LLMs vs Classical Algorithms in Maze Pathfinding

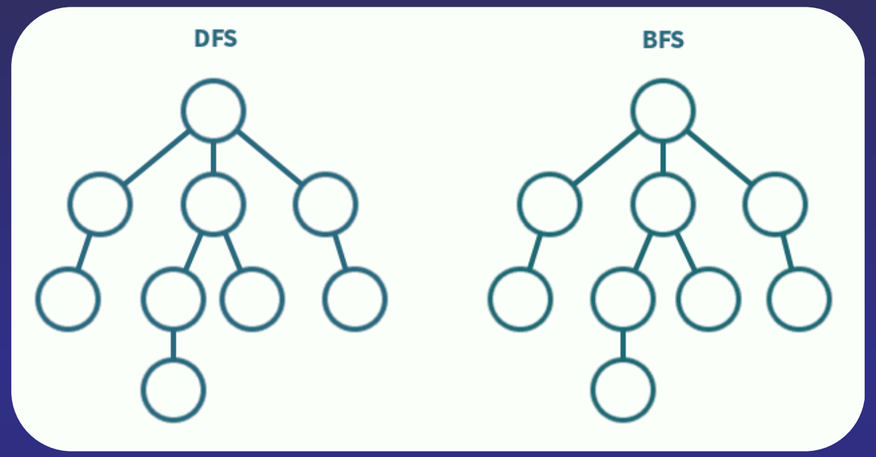
GitHub repository

Ainur Suranchina
AI Research Project

Problem & Motivation

- Large Language Models are widely used for many tasks
- They require significant computational resources
- Many problems already have efficient algorithmic solutions
- Is it reasonable to use LLMs for structured tasks?

BFS / DFS / A*



vs

AI agents

Research Question

To what extent are LLMs effective, reliable, and efficient compared to classical algorithms in solving maze problems?

Methodology

10 randomly generated mazes (10×10)

Algorithms: BFS, DFS, A*

Models: LLaMA 8B, LLaMA 70B, GPT-OSS 20B

Metrics:
- Path length
- Success rate
- Execution time

Example of maze

```
Maze:
S 0 1 1 1 1 1 1 1 1 1
1 0 0 0 1 0 0 0 0 0 1
1 1 0 0 1 0 1 0 1 0 1
1 0 1 0 1 0 1 0 1 0 1
1 0 1 0 1 1 1 0 1 0 1
1 0 1 0 1 0 0 0 0 0 1
1 0 1 0 1 0 1 1 1 0 1
1 0 1 0 0 0 1 0 0 0 1
1 0 1 1 1 1 0 0 1 1 1
1 0 0 0 0 0 0 0 0 0 1
1 1 1 1 1 1 1 1 1 0 G
```

Algorithm Behavior

```
Combined Paths (BFS=B, DFS=D, A*=A):

S B 1 1 1 1 1 1 1 1 1
1 B B D 1 0 0 0 0 0 1
1 1 B B 1 0 1 0 1 0 1
1 0 1 B 1 0 1 0 1 0 1
1 0 1 B 1 1 1 0 1 0 1
1 0 1 B 1 B B B B B 1
1 0 1 B 1 B 1 1 1 B 1
1 0 1 B B B 1 B B B 1
1 0 1 1 1 1 D B 1 1 1
1 0 0 0 0 0 D B B B 1
1 1 1 1 1 1 1 1 1 B G

Lengths:
BFS: 29
DFS: 31
A* : 29
```

Combined Paths (B, D, A)

- BFS and A* always find the shortest path
- DFS explores deeper and produces longer paths

Algorithm Results

- BFS: optimal in all cases
- A*: optimal in all cases
- DFS: up to 2x longer paths

GitHub repository

maze_id	bfs_length	dfs_length	astar_length	bfs_time	dfs_time	astar_time
0	29	31	29	0.00012350082397460938	0.00012373924255371094	0.00014162063598632812
1	21	23	21	9.822845458984375e-05	0.00010085105895996094	8.130073547363281e-05
2	21	35	21	0.00010156631469726562	0.00010633468627929688	0.00015139579772949 22
3	21	21	21	8.702278137207031e-05	8.249282836914062e-05	0.00012540817260742188
4	29	37	29	0.00010848045349121094	9.942054748535156e-05	0.00016689300537109375
5	21	45	21	0.00010275840759277344	9.632110595703125e-05	6.67572021484375e-05
6	21	29	21	8.749961853027344e-05	0.00010776519775390625	7.128715515136719e-05
7	25	43	25	0.00010323524475097656	8.749961853027344e-05	9.846687316894531e-05
8	29	33	29	9.679794311523438e-05	0.00010013580322265625	0.00012946128845214844
9	21	49	21	9.512901306152344e-05	0.00010704994201660156	7.677078247070312e-05

(LLaMA 8B, LLaMA 70B, GPT-OSS 20B and result comparison are in the next slides)

LLM Results

LLaMA 8B Results:

- Success rate: 0%
- Common errors:
  - hitting walls
  - loops
  - incomplete paths

```
(1,2) -> DOWN -> (2,2)
Current coordinates: (2, 2)
Current position: 0

(2,2) -> DOWN -> (3,2)
Current coordinates: (3, 2)
Current position: 0

(3,2) -> DOWN -> (4,2)
Current coordinates: (4, 2)
Current position: 1 (Blocked by wall, cannot move down)

(3,2) -> RIGHT -> (4,2)
Current coordinates: (4, 2)
Current position: 1 (Blocked by wall, cannot move right)

(3,2) -> DOWN -> (4,2)
Current coordinates: (4, 2)
Current position: 1 (Blocked by wall, cannot move down)

(3,2) -> LEFT -> (3,1)
Current coordinates: (3, 1)
Current position

Parsed moves: []
LLM Path length: Invalid path
BFS Path length: 29
```

LLaMA 70B Results:

- Success rate: 0%
- Better reasoning than 8B
- Often close to solution but fails at the end

```
LLM raw output: (0,0) -> RIGHT -> (0,1)
(0,1) -> RIGHT -> (0,2) is not possible because there is a wall, so we go DOWN
(0,1) -> DOWN -> (1,1)
(1,1) -> RIGHT -> (1,2)
(1,2) -> RIGHT -> (1,3)
(1,3) -> RIGHT -> (1,4) is not possible because there is a wall, so we go DOWN
(1,3) -> DOWN -> (2,3)
(2,3) -> RIGHT -> (2,4) is not possible because there is a wall, so we go DOWN
(2,3) -> DOWN -> (3,3)
(3,3) -> LEFT -> (3,2)
(3,2) -> UP -> (2,2)
(2,2) -> RIGHT -> (2,3)
(2,3) -> RIGHT -> (2,4) is not possible because there is a wall, so we go DOWN
(2,3) -> DOWN -> (3,3)
(3,3) -> DOWN -> (4,3)
(4,3) -> DOWN -> (5,3)
(5,3) -> DOWN -> (6,3)
(6,3) -> DOWN -> (7,3)
(7,3) -> DOWN -> (8,3)
(8,3) -> DOWN -> (9,3)
(9,3) -> DOWN -> (10,3)
(10,3) -> RIGHT -> (10,4)
(10,4) -> RIGHT -> (10,5)
(10,5) -> RIGHT -> (10,6)
(10,6) -> RIGHT -> (10,7)
(10,7) -> RIGHT -> (10,8)
(10,8) ->

Parsed moves: []
LLM Path length: Invalid path
BFS Path length: 29
```

GPT-OSS 20B Results:

- Success rate: 60%
- Sometimes finds optimal path
- Much slower than algorithms

```
From (0,0) to (0,1): RIGHT
(0,1) to (1,1): DOWN
(1,1) to (1,2): RIGHT
(1,2) to (1,3): RIGHT
(1,3) to (2,3): DOWN
(2,3) to (3,3): DOWN
(3,3) to (4,3): DOWN
(4,3) to (5,3): DOWN
(5,3) to (6,3): DOWN
(6,3) to (7,3): DOWN
(7,3) to (7,4): RIGHT
(7,4) to (7,5): RIGHT
(7,5) to (6,5): UP
(6,5) to (5,5): UP? Wait from 6,5 to 5,5 is UP. Actually earlier I wrote down but it's up. Let's double-check: 6,5 -> 5,5 is up. So we need UP.
(5,5) to (5,6): RIGHT
(5,6) to (5,7): RIGHT
(5,7) to (5,8): RIGHT
(5,8) to (5,9): RIGHT
(5,9) to (6,9): DOWN
(6,9) to (7,9): DOWN
(7,9) to (7,8): LEFT
(7,8) to (7,7): LEFT
(7,7) to (8,7): DOWN
(8,7) to (9,7): DOWN
(9,7) to (9,8): RIGHT
(9,8) to (9,9): RIGHT
(9,9) to (10,9): DOWN
(10,9) to (10,10): RIGHT

RIGHT, DOWN, RIGHT, RIGHT, DOWN, DOWN, DOWN, DOWN, DOWN, DOWN, RIGHT, RIGHT, UP, UP, RIGHT, RIGHT, RIGHT, RIGHT, DOWN, DOWN, LEFT, LEFT, DOWN, DOWN, RIGHT, RIGHT, DOWN, RIGHT
```

Time Comparison

- Algorithms: ~0.0001 seconds
- LLMs: 3–30 seconds

LLMs are up to 100,000× slower

Method	Success Rate	Optimal	Time
BFS	100%	Yes	~0.0001 s
A*	100%	Yes	~0.0001 s
DFS	100%	No	~0.0001 s
LLaMA 8B	0%	No	1-3 s
LLaMA 70B	0%	No	1-3 s
GPT-OSS 20B	60%	Sometimes	3–30 s

```
maze_id,bfs_length,dfs_length,astar_length,llm_length,llm_success,bfs_time,dfs_time,astar_time,llm_time
0,29,31,29,0,False,0.00015401840209960938,0.0001475811004638672,0.00016045570373535156,6.7719457149505615
1,21,23,21,21,True,0.00036263465881347656,0.0003058910369873047,0.00022149085998535156,3.4035451412200928
2,21,35,21,25,True,0.00035190582275390625,0.00029206275939941406,0.0004696846008300781,6.612376928329468
3,21,21,21,21,True,0.00019550323486328125,0.00015592575073242188,0.00025200843811035156,29.92789101600647
4,29,37,29,0,False,0.00029277801513671875,0.0002682209014892578,0.0004630088806152344,17.75356388092041
```

GPT-OSS 20b and algorythms comparison
You can find more in Github repository or Research paper (in good quality)

Key Findings and why LLMs Fail

- Algorithms are:
  - reliable
  - fast
  - optimal

- LLMs are:
  - inconsistent
  - slow
  - sometimes incorrect

- No strict state tracking
- Probabilistic reasoning
- No guarantee of correctness

Conclusion

- Classical algorithms outperform LLMs
- LLMs are not suitable for structured tasks
- Simpler solutions are often more efficient

Thank You!

Thank you for your attention!

GitHub repository

Questions?

Telegram: @arafnel
Email: miss.smile135@gmail.com
+77082762185